

Florence: Infrastructure - GitHub Actions

1. Overview and Purpose

This document outlines the Continuous Integration and Continuous Deployment (CI/CD) pipeline configured via GitHub Actions. The workflow automates the build and distribution process for the Florence mobile application to ensure that updates are delivered to testers and production environments efficiently. The configuration is located in `.github/workflows/main.yml`.

2. Workflow Triggers

The deployment pipeline is automatically initiated when code is pushed to the `production` branch. It can also be triggered manually via the GitHub Actions interface using the `workflow_dispatch` event. All operations are scoped to the `florence/platform_service` directory.

3. Environment Provisioning

Before evaluating the deployment strategy the workflow provisions an Ubuntu environment equipped with JDK 17, the stable Flutter channel and Shorebird. It utilises caching to speed up subsequent runs and executes `flutter pub get` to resolve all project dependencies.

4. The Smart Decision Logic

The workflow features an intelligent decision-making step that determines whether to perform a full application release or an over-the-air patch.

- It extracts the current version string from `pubspec.yaml`.
- It compares this value against the version from the previous commit using a full git history fetch.

- If the version number has changed the workflow executes a Full Release.
 - If the version number remains identical the workflow executes a Shorebird Patch.
-

5. Deployment Scenarios

Scenario A: Full Release

When a version change is detected, the workflow compiles a new Android APK using Shorebird.

- It injects the live production backend URLs via compile-time arguments.
- It uploads the compiled binary to Firebase App Distribution.
- The build is automatically distributed to the designated testers group for quality assurance and client review.

Scenario B: OTA Patch

When no version change is detected, the workflow utilises Shorebird to push an over-the-air patch.

- This updates the Dart and Flutter code on existing installations instantly without requiring users to download a new APK.
- It injects the identical production backend URLs to maintain environment consistency.

Note: The current automation pipeline is strictly configured for Android APK generation and patching. iOS deployment requires separate Apple Developer credentials and macOS runners which are not currently integrated into this specific workflow.

6. Backend Configuration Injection

During both release and patch scenarios the workflow injects the live microservice URLs directly into the Flutter application. This guarantees the mobile app communicates exclusively with the correct production environments:

- **Data Service:** `https://ds-florence-dhp.vercel.app`
- **LLM Chatbot Service:** `https://llmcs-florence-dhp.vercel.app`
- **LLM Engine Service:** `https://llmes-florence-dhp.vercel.app`

7. Secrets and Credentials

The following repository secrets are strictly required for the automation pipeline to function. These are managed under the repository Settings and are never exposed in the source code.

- `SHOREBIRD_TOKEN` : Authenticates the Shorebird CLI for releasing binaries and pushing patches.
 - `FIREBASE_APP_ID` : Identifies the specific Firebase project for App Distribution routing.
 - `CREDENTIAL_FILE_CONTENT` : A base64 encoded Firebase service account JSON key that grants GitHub Actions permission to upload builds securely.
-

8. Day 2 Operations and Maintenance

- **Deploying a Major or Minor Update:** The version number in `florence/platform_service/pubspec.yaml` must be incremented before merging code into the `production` branch. The pipeline will automatically recognise the change and distribute a fresh APK via Firebase.
- **Deploying a Hotfix or UI Tweak:** Code is merged into `production` without altering the version number. The pipeline will automatically route to the patch scenario and update live devices instantly.
- **Manual Triggers:** If a rebuild is required without any code changes the pipeline can be manually dispatched from the Actions tab on GitHub.